

Yes, for a browser game like Particul, I would not use only React.

Particul is an incremental/idler game with click-to-mine particles, particles falling to the bottom, automation, selling, upgrades, research, trading, and rare particles. Steam tags it as Incremental, Idler, Physics, Mining, Pixel Graphics, Automation, 2D.

For a browser version with a mandatory Java backend, I'd use:

Frontend:

Phaser 3 + TypeScript + Vite

UI overlay:

React optional, but not required at the start

Backend:

Java Spring Boot

Database:

PostgreSQL

Best frontend choice: Phaser

For this specific style, Phaser is the right frontend/game framework because you need:

particles falling

click mining

2D physics-like movement

sprites

animations

object pooling

floating numbers

game loop

effects

React alone is not ideal for thousands of moving particles. React is better for menus and panels, not rendering a physics-like particle pile.

Recommended architecture

Browser

├─ Phaser game canvas

| └─ particle spawning

| └─ falling particles

| └─ click effects

| └─ extractors

| └─ animations

|

├─ UI panels

| └─ upgrades

| └─ lab/research

| |— traders
| |— marketplace
| |— prestige/ascend
|

Java Spring Boot backend

|— login/register
|— save/load player state
|— validate purchases
|— calculate offline progress
|— leaderboards
|— anti-cheat

Communication: REST first, not WebSocket first

For a Particul-like game, I would still start with REST.

Use REST for:

GET /api/game/state
POST /api/game/click
POST /api/game/buy-extractor
POST /api/game/sell-particles
POST /api/game/research
POST /api/game/trade
POST /api/game/ascend

Use WebSocket only later for:

live leaderboard
global events
chat
market price changes
multiplayer trading
server announcements
Important design rule

The browser should handle the visual particles, but the backend should handle the real economy.

Good design:

Phaser shows:

- particles falling
- animations
- estimated income
- nice effects

Spring Boot calculates:

- real particle count

- upgrades owned
- production per second
- offline gains
- purchase validation

Do not let the frontend say:

"I earned 999999 particles"

Instead, frontend says:

"I clicked"

"I want to buy upgrade"

"I want to sell particles"

Then the Java backend calculates whether that is valid.

My final recommendation

For a browser game inspired by Particul:

Phaser 3 + TypeScript + Vite

Java Spring Boot

PostgreSQL

REST API first

WebSocket later only if needed

React can be added later for complex menus, but I would start with Phaser + TypeScript first.